

Decentralized Query processing and Optimization in Apache Spark

Εργασία στο Μάθημα Τεχνολογίες Αποκεντρωμένων Δεδομένων

2023-2024



Μέλη Ομάδας

Αμίσης Ελευθέριος, 1072464

Παπαχριστοφίλου Σαράντης, 1072600

Θέμα 1

Στο συγκεκριμένο θέμα καλούμαστε να διαχειριστούμε δύο αρχεία, τα οποία περιέχουν μετρήσεις για τη θερμοκρασία (σε °C) και την υγρασία (σε ποσοστό %) σε μία ευρωπαϊκή πόλη για την ίδια χρονική περίοδο.

Ξεκινάμε με την εισαγωγή των απαραίτητων βιβλιοθηκών που θα μας χρειαστούν στην συνέχεια. Εισάγεται το module **functions** και ένα σέτ από τις συναρτήσεις που περιέχει: **col, avg, month, sqrt, sum**. Οι συναρτήσεις αυτές χρησιμοποιούνται αργότερα για τους υπολογισμούς που απαιτούν τα ερωτήματα. Επίσης, εισάγεται το module **pyspark.sql.types** με την κλάση **FloatType**, ώστε να διαχειριστούμε τους δεκαδικούς αριθμούς των μετρήσεων.

Cmd 1

Απαραίτητες Βιβλιοθήκες

Cmd 2

```
1 from pyspark.sql import functions
2 from pyspark.sql.functions import col, avg, month, sqrt, sum
3 from pyspark.sql.types import FloatType
```

Συνεχίζουμε με το διάβασμα του αρχείου tempm.txt που περιέχει τις τιμές της θερμοκρασίας και την δημιουργία του αντίστοιχου dataframe. Ορίζουμε το path του αρχείου όπως αυτό δημιουργήθηκε αφού το φορτώσαμε στο databricks και το διαβάζουμε δημιουργώντας ένα αρχικό dataframe, το οποίο αποτελείται από μία στήλη και σε κάθε γραμμή περιέχει όλες τις μετρήσεις που αφορούν μία συγκεκριμένη ημερομηνία.

Cmd 3

Δημιουργία Dataframe για την Θερμοκρασία

Cmd 4

```
1 # Path για το αρχείο tempm.txt
2 tempm_file_path = "dbfs:/FileStore/tables/tempm.txt"
3
4 # Διάβασμα Αρχείου και Δημιουργία dataframe
5 tempm_df = spark.read.text(tempm_file_path)
6
7 # Parse το dataframe σε ξεχωριστές στήλες
8 tempm_df = tempm_df.selectExpr("CAST(value AS STRING) AS json_str")
9 tempm_df = tempm_df.selectExpr("from_json(json_str, 'MAP<STRING, STRING>') AS data")
10 tempm_df = tempm_df.selectExpr("explode(data) AS (timestamp, temperature)")
11
12 # Μετατροπή του column temperature σε float
13 tempm_df = tempm_df.withColumn("temperature", col("temperature").cast(FloatType()))
14
15 tempm_df.show()
16 #display(tempm_df)
```

Το περιεχόμενο της στήλης value το μετατρέπουμε σε string και την στήλη αυτή την μετονομάζουμε σε json_str. Κάνουμε parse τα strings και δηλώνουμε πως κάθε στήλη περιέχει δύο στοιχεία, ένα κλειδί και μία τιμή, τύπου string. Έτσι, προκύπτει το τελικό dataframe με δύο στήλες, τις timestamp και temperature, με τις τιμές στην στήλη temperature να μετατρέπονται σε float.

```

tempm_df
+-----+-----+
|          timestamp|temperature|
+-----+-----+
|2014-02-13T06:20:00|          3.0|
|2014-02-13T13:50:00|          7.0|
|2014-02-13T06:00:00|          2.0|
|2014-02-13T03:00:00|          3.0|
|2014-02-13T13:00:00|          6.0|
|2014-02-13T18:50:00|          4.0|
|2014-02-13T13:20:00|          6.0|
|2014-02-13T15:00:00|          6.0|
|2014-02-13T08:50:00|          4.0|
|2014-02-13T21:50:00|          4.0|
|2014-02-13T08:00:00|          3.0|
|2014-02-13T07:50:00|          3.0|
|2014-02-13T08:20:00|          4.0|
|2014-02-13T21:20:00|          3.0|
|2014-02-13T11:50:00|          6.0|
|2014-02-13T11:20:00|          6.0|
|2014-02-13T17:50:00|          5.0|
|2014-02-13T11:00:00|          6.0|
|2014-02-13T05:50:00|          2.0|
|2014-02-13T20:50:00|          3.0|
+-----+-----+
only showing top 20 rows

```

Ακολουθεί η δημιουργία του αντίστοιχου dataframe για τις μετρήσεις που αφορούν την υγρασία, το ίδιο χρονικό διάστημα. Η διαδικασία που ακολουθείται είναι ακριβώς η ίδια με αυτήν της δημιουργίας του tempm_df. Ομοίως, λοιπόν, διαβάζεται το αρχείο hum.txt, αναλύονται τα κλειδιά και οι τιμές από το αρχικό dataframe και προκύπτει το τελικό με δύο στήλες. Οι στήλες αυτές είναι οι timestamp και humidity, με τις τιμές της στήλης humidity να μετατρέπονται σε float.

Δημιουργία Dataframe για την Υγρασία

```

1  # Path για το αρχείο hum.txt
2  hum_file_path = "dbfs:/FileStore/tables/hum.txt"
3
4  # Διάβασμα Αρχείου και Δημιουργία dataframe
5  hum_df = spark.read.text(hum_file_path)
6
7  # Parse το dataframe σε ξεχωριστές στήλες
8  hum_df = hum_df.selectExpr("CAST(value AS STRING) AS json_str")
9  hum_df = hum_df.selectExpr("from_json(json_str, 'MAP<STRING, STRING>') AS data")
10 hum_df = hum_df.selectExpr("explode(data) AS (timestamp, humidity)")
11
12 # Μετατροπή του column humidity σε float
13 hum_df = hum_df.withColumn("humidity", col("humidity").cast(FloatType()))
14
15 hum_df.show()
16 #display(hum_df)

```

<u>hum_df</u>	
timestamp	humidity
2014-02-13T06:20:00	93.0
2014-02-13T13:50:00	66.0
2014-02-13T06:00:00	91.0
2014-02-13T03:00:00	84.0
2014-02-13T13:00:00	62.0
2014-02-13T18:50:00	75.0
2014-02-13T13:20:00	70.0
2014-02-13T15:00:00	56.0
2014-02-13T08:50:00	87.0
2014-02-13T21:50:00	75.0
2014-02-13T08:00:00	88.0
2014-02-13T07:50:00	93.0
2014-02-13T08:20:00	87.0
2014-02-13T21:20:00	81.0
2014-02-13T11:50:00	76.0
2014-02-13T11:20:00	76.0
2014-02-13T17:50:00	70.0
2014-02-13T11:00:00	69.0
2014-02-13T05:50:00	93.0
2014-02-13T20:50:00	81.0

only showing top 20 rows

- **Ερώτημα 1ο:** Σε πόσες μέρες η θερμοκρασία κυμάνθηκε από 18 °C έως 22 °C;

Για να βρούμε τις ημέρες που η θερμοκρασία κυμάνθηκε από 18 °C έως 22 °C πρώτα θα φιλτράρουμε το dataframe για κρατήσουμε τις γραμμές στις οποίες η θερμοκρασία βρίσκεται σε αυτό το διάστημα. Επειδή, ωστόσο, θέλουμε να μετρήσουμε διακριτές ημερομηνίες και όχι τιμές που αφορούν την ίδια ημέρα, κρατάμε από την στήλη timestamp μόνο ένα πεδίο date που περιέχει ημερομηνίες χωρίς την ώρα μέτρησης. Ανάμεσα στις ημερομηνίες που έχουν απομείνει υπάρχουν αρκετές ίδιες. Συνεπώς, μέσω της distinct() κρατάμε μόνο τις διακριτές ημερομηνίες και μέσω της len υπολογίζουμε το μήκος του dataframe που αποτελεί και το σύνολο των ημερών, στις οποίες η θερμοκρασία κυμάνθηκε από 18 °C έως 22 °C.

Cmd 7

Σε πόσες μέρες η θερμοκρασία κυμάνθηκε από 18 βαθμούς Κελσίου έως 22 βαθμούς Κελσίου;

Cmd 8

```
1 # Παίρνουμε μόνο τις γραμμές με θερμοκρασία στο διάστημα [18,22]
2 filtered_tempm_df = tempm_df.filter((col("temperature") >= 18) & (col("temperature") <= 22))
3
4 # Dataframe με ημερομηνία μόνο αντί για timestamp
5 filtered_tempm_df = filtered_tempm_df.withColumn("date", col("timestamp").cast("date"))
6
7 # Επιλέγουμε διακριτές ημερομηνίες από το filtered_temp_df
8 distinct_dates = filtered_tempm_df.select("date").distinct().collect()
9 num_of_days = len(distinct_dates)
10
11 # ΑΠΑΝΤΗΣΗ
12 print("Σε πόσες μέρες η θερμοκρασία κυμάνθηκε από 18 °C έως 22 °C;")
13 print("ΑΠΑΝΤΗΣΗ:", num_of_days)
```

Απάντηση

Σε πόσες μέρες η θερμοκρασία κυμάνθηκε από 18 °C έως 22 °C;
ΑΠΑΝΤΗΣΗ: 26

- **Ερώτημα 2ο:** Ποιες ήταν οι 10 πιο κρύες μέρες και οι 10 πιο ζεστές μέρες;

Η ταξινόμηση των ημερών σε πιο ζεστές και πιο κρύες γίνεται βάσει της μέσης θερμοκρασίας κάθε ημέρας και όχι βάσει των ακραίων τιμών. Συνεπώς, για εύκολο υπολογισμό των δεδομένων στο αρχικό tempm_df προσθέτουμε μία νέα στήλη "date" η οποία περιέχει μόνο την ημερομηνία της κάθε μέτρησης. Στην συνέχεια, κάνουμε group_by κάθε μέτρηση βάσει της ημερομηνίας της και μέσω της avg υπολογίζουμε τον μέσο όρο για κάθε date. Όταν απαντάμε στο ερώτημα των πιο κρύων ημερών γίνεται ταξινόμηση των τιμών κατά αύξουσα σειρά και κρατάμε μέσω της limit(10) μόνο τα δέκα πρώτα αποτελέσματα, ενώ όταν

απαντάμε στο ερώτημα των πιο ζεστών ημερών, η ταξινόμηση είναι φθίνουσα (ascending=False) και κρατάμε με τον ίδιο τρόπο τα δέκα πρώτα αποτελέσματα.

Cmd 9

Ποιες ήταν οι 10 πιο κρύες μέρες και οι 10 πιο ζεστές μέρες;

Cmd 10

```
1  dates_df = tempm_df.withColumn("date", col("timestamp").cast("date"))
2
3  # 10 πιο Κρύες Μέρες Βάσει Μέσης Θερμοκρασίας
4  coldest_days_avg = dates_df.groupBy("date").agg(avg("temperature").alias("min_temperature")).orderBy("min_temperature").limit(10)
5
6  # 10 πιο Ζεστές Μέρες Βάσει Μέσης Θερμοκρασίας
7  hottest_days_avg = dates_df.groupBy("date").agg(avg("temperature").alias("max_temperature")).orderBy("max_temperature", ascending=False).limit(10)
8
9  # ΑΠΑΝΤΗΣΗ
10 print("Ποιες ήταν οι 10 πιο κρύες μέρες;")
11 coldest_days_avg.show(truncate=False)
12
13 print("Ποιες ήταν οι 10 πιο ζεστές μέρες;")
14 hottest_days_avg.show(truncate=False)
```

Απάντηση

Ποιες ήταν οι 10 πιο κρύες μέρες;

date	min_temperature
2014-03-27	2.8260869565217392
2014-02-14	3.391304347826087
2014-03-02	3.857142857142857
2014-03-11	3.9027777777777777
2014-03-01	4.083333333333333
2014-02-13	4.1
2014-03-03	4.132075471698113
2014-03-25	4.152777777777778
2014-03-05	4.211267605633803
2014-02-17	4.211267605633803

Ποιες ήταν οι 10 πιο ζεστές μέρες;

date	max_temperature
2014-05-22	20.0
2014-05-21	17.638888888888889
2014-06-08	17.638888888888889
2014-05-23	16.257142857142856
2014-06-02	16.056338028169016
2014-05-24	15.833333333333334
2014-06-03	15.830985915492958
2014-06-04	15.710144927536232
2014-06-07	15.666666666666666
2014-05-25	15.619718309859154

- **Ερώτημα 3ο: Ποιος μήνας είχε την υψηλότερη τυπική απόκλιση στις τιμές της υγρασίας;**

Για τον υπολογισμό της τυπικής απόκλισης ακολουθούμε την κλασική μεθοδολογία, η οποία ξεκινά με την εύρεση της μέσης τιμής της υγρασίας για κάθε μήνα, στο νέο dataframe `avg_hum`. Επίσης, προστίθεται μία νέα στήλη, η οποία δηλώνει τον μήνα στον οποίο ανήκει κάθε γραμμή.

Το επόμενο βήμα, περιλαμβάνει τον υπολογισμό της διαφοράς των τετραγώνων μεταξύ μίας τιμής υγρασίας και της μέσης τιμής υγρασίας του μήνα στον οποίο ανήκει η συγκεκριμένη μέτρηση. Για να συμβεί αυτό δημιουργείται το dataframe “`diafores_tetragwnwn`” με join του `avg_hum` στο dataframe `hum_df` και περιέχει μία νέα στήλη “`diafores_tetragwnwn`”.

Αφού, υπολογιστούν οι διαφορές τετραγώνων, ακολουθεί το άθροισμα των διαφορών τετραγώνων για κάθε μήνα. Χρησιμοποιείται, η `group_by` για να ομαδοποιήσουμε τις διαφορές ανάλογα τον μήνα και η `sum` για τον σχηματισμό των αθροισμάτων, στο νέο dataframe “`sum_diafwrwn_tetragwnwn`”.

Στο επόμενο join τώρα, το dataframe “`num_rows_per_month`” περιέχει το πλήθος των ημερών για κάθε μήνα, προκειμένου να γίνει η διαίρεση και ο υπολογισμός της διακύμανσης. Εδώ γίνεται ταυτόχρονα η διαίρεση και ο υπολογισμός της ρίζας του πηλίκου, άρα βρίσκουμε τις τυπικές αποκλίσεις όλων των μηνών.

Για την απάντηση ταξινομούμε αυτές τις τιμές σε φθίνουσα σειρά και κρατάμε μόνο το 1ο αποτέλεσμα.

Cmd 12

Ποιος μήνας είχε την υψηλότερη τυπική απόκλιση στις τιμές της υγρασίας;

Cmd 13

```
1  # Μέση Υγρασία Κάθε Μήνα
2  avg_hum = hum_df.groupBy(month("timestamp").alias("month")).agg(avg("humidity").alias("avg_humidity"))
3
4  # Προσθήκη Στήλης Month στο hum_df
5  hum_df = hum_df.withColumn("month", month("timestamp"))
6
7  # Διαφορές Τετραγώνων
8  diafores_tetragwnwn = (
9      hum_df.join(avg_hum, "month")
10     .withColumn("diafores_tetragwnwn", (col("humidity") - avg_hum["avg_humidity"])**2)
11     .select("month", "timestamp", "humidity", "avg_humidity", "diafores_tetragwnwn")
12 )
13
14 # Άθροισμα Διαφορών Τετραγώνων Κάθε Μήνα
15 sum_diafwrwn_tetragwnwn = (
16     diafores_tetragwnwn
17     .groupBy("month")
18     .agg(sum("diafores_tetragwnwn").alias("Άθροισμα Διαφορών Τετραγώνων"))
19     .select(col("month").alias("Μήνας"), col("Άθροισμα Διαφορών Τετραγώνων"))
20 )
21
22 # Αριθμός Μετρήσεων Κάθε Μήνα
23 num_rows_per_month = hum_df.groupBy("month").count()
```

```

24
25 # Υπολογισμός Τυπικής Απόκλισης
26 tupiki_apoklisi = (
27     sum_diafwrwn_tetragwnwn
28     .join(num_rows_per_month, sum_diafwrwn_tetragwnwn["Μήνας"] == num_rows_per_month["month"])
29     .withColumn("tupiki_apoklisi", sqrt(col("Αθροισμα Διαφορών Τετραγώνων") / col("count")))
30     .select("Μήνας", col("tupiki_apoklisi").alias("Τυπική Απόκλιση"))
31 )
32
33 #ΑΠΑΝΤΗΣΗ
34 max_ta = tupiki_apoklisi.orderBy(functions.desc("Τυπική Απόκλιση")).limit(1)
35 print("Ποιος μήνας είχε την υψηλότερη τυπική απόκλιση στις τιμές της υγρασίας;")
36 max_ta.show(truncate=False)

```

Απάντηση

Ποιος μήνας είχε την υψηλότερη τυπική απόκλιση στις τιμές της υγρασίας;

```

+-----+-----+
|Μήνας|Τυπική Απόκλιση|
+-----+-----+
|4     |17.72090118932502|
+-----+-----+

```

- **Ερώτημα 4ο:** Ποια ήταν η μέγιστη και ποια η ελάχιστη τιμή του δείκτη δυσφορίας;

Ο υπολογισμός της ελάχιστης και της μέγιστης τιμής του δείκτη δυσφορίας γίνεται σύμφωνα με τον τύπο: **DI = T - 0,55 (1 - 0,01*RH) (T - 14,5)**. Αρχικά, κάνουμε join το hum_df στο tempm_df και δημιουργούμε ένα νέο dataframe, το DI_upologismos. Έτσι, για κάθε timestamp στο οποίο έχει γίνει μέτρηση έχουμε τις τιμές για την υγρασία και την θερμοκρασία. Timestamps με NULL τιμές βγαίνουν από το dataframe, ενώ δημιουργείται μία νέα στήλη η "DI", στην οποία τοποθετούνται οι τιμές του δείκτη δυσφορίας σύμφωνα με τον παραπάνω τύπο.

Όσον αφορά την εύρεση της μέγιστης και της ελάχιστης τιμής γίνεται ταξινόμηση κατά φθίνουσα και κατά αύξουσα σειρά αντίστοιχα, κρατώντας σε κάθε περίπτωση μόνο το πρώτο αποτέλεσμα.

Cmd 14

Ποια ήταν η μέγιστη και ποια η ελάχιστη τιμή του δείκτη δυσφορίας;

DI = T - 0,55 (1 - 0,01*RH) (T - 14,5) , T: θερμοκρασία, RH: υγρασία

```

1  # Συνδυασμός dataframe υγρασίας, θερμοκρασίας και υπολογισμός DI
2  DI_upologismos = (
3      tempm_df
4      .join(hum_df, tempm_df["timestamp"] == hum_df["timestamp"])
5      .na.drop(subset=["temperature", "humidity"]) # Βγάζουμε τις NULL τιμές
6      .withColumn("DI", col("temperature")-0.55*(1-0.01*col("humidity"))*(col("temperature")-14.5))
7      .select(tempm_df["timestamp"], "temperature", "humidity", "DI")
8  )
9
10 # Μέγιστο και Ελάχιστο DI
11 max_DI = DI_upologismos.orderBy(col("DI").desc()).limit(1).select("DI")
12 min_DI = DI_upologismos.orderBy(col("DI").asc()).limit(1).select("DI")
13
14 print("Η ελάχιστη τιμή του δείκτη δυσφορίας είναι: ")
15 min_DI.show(truncate=False)
16 print("Η μέγιστη τιμή του δείκτη δυσφορίας είναι: ")
17 max_DI.show(truncate=False)

```

Απάντηση

Η ελάχιστη τιμή του δείκτη δυσφορίας είναι:

```

+-----+
|DI      |
+-----+
|-2.3262500000000004|
+-----+

```

Η μέγιστη τιμή του δείκτη δυσφορίας είναι:

```

+-----+
|DI      |
+-----+
|22.1125|
+-----+

```

Θέμα 2

Στο συγκεκριμένο θέμα καλούμαστε να διαχειριστούμε τρία διαφορετικά αρχεία με το καθένα να περιέχει πλήρη ιστορικά δεδομένα για τις ημερήσιες τιμές τριών μετοχών από το 2005 έως το 2017.

Ξεκινάμε με την εισαγωγή των απαραίτητων βιβλιοθηκών που θα μας χρειαστούν στην συνέχεια. Εισάγεται το module **functions** και ένα σετ από τις συναρτήσεις που περιέχει: **col, avg, month, year, date_format, max, min, format_number, year**. Οι συναρτήσεις αυτές χρησιμοποιούνται αργότερα για τους υπολογισμούς που απαιτούν τα ερωτήματα. Επίσης, εισάγεται το module **pyspark.sql.types** με τις κλάσεις **FloatType** και **StringType** ώστε να διαχειριστούμε τις διάφορες τιμές των μετοχών.

Cmd 1

Απαραίτητες Βιβλιοθήκες

Cmd 2

```
1 from pyspark.sql import functions
2 from pyspark.sql.types import StringType, FloatType
3 from pyspark.sql.functions import col, avg, month, year, date_format, max, min, format_number, year
```

Τα αρχεία που περιέχουν πληροφορίες για τις μετοχές είναι σε csv μορφή, οπότε αρκεί να τα φορτώσουμε στο databricks, να ορίσουμε το μονοπάτι τους και μέσω της `spark.read.csv` να δημιουργήσουμε ένα dataframe για κάθε αρχείο.

Συγκεκριμένα, δημιουργούμε τα `agn_df`, `ainv_df` και `ale_df`.

Όταν διαβάζουμε τα αρχεία, τα στοιχεία τους μπαίνουν στο dataframe ως strings και όχι ως αριθμοί. Για τον λόγο αυτό καλούμαστε να μετατρέψουμε και στα τρία παραπάνω dataframes το περιεχόμενο των στηλών `Open`, `High`, `Low`, `Close`, `Volume` και `OpenInt` σε `Float` από `String`.

Cmd 4

```
1 # Path για το αρχείο
2 agn_file_path = "dbfs:/FileStore/tables/agn_us.txt"
3 ainv_file_path = "dbfs:/FileStore/tables/ainv_us.txt"
4 ale_file_path = "dbfs:/FileStore/tables/ale_us.txt"
5
6 # Διάβασμα Αρχείων και Δημιουργία dataframe για τα τρία αρχεία
7 agn_df = spark.read.csv(agn_file_path, header=True)
8 ainv_df = spark.read.csv(ainv_file_path, header=True)
9 ale_df = spark.read.csv(ale_file_path, header=True)
10
11 # Κάνουμε τα παρακάτω στήλες τύπου float από string
12 float_columns = ["Open", "High", "Low", "Close", "Volume", "OpenInt"]
13 for column in float_columns:
14     agn_df = agn_df.withColumn(column, agn_df[column].cast(FloatType()))
15     ainv_df = ainv_df.withColumn(column, ainv_df[column].cast(FloatType()))
16     ale_df = ale_df.withColumn(column, ale_df[column].cast(FloatType()))
17
18 display(agn_df)
19 display(ainv_df)
20 display(ale_df)
```

agn_df

	Date	Open	High	Low	Close	Volume	OpenInt
1	2005-01-03	32.31	32.31	31.527	31.616	1027044	0
2	2005-01-04	31.527	31.616	31.22	31.338	1927762	0
3	2005-01-05	30.971	31.051	30.714	30.843	943399	0
4	2005-01-06	30.843	31.398	30.764	31.26	662398	0
5	2005-01-07	31.26	31.26	30.456	30.566	1087886	0
6	2005-01-10	30.506	31.23	30.486	30.863	896381	0
7	2005-01-11	30.843	30.903	30.199	30.377	835841	0

↓ 3,239 rows | 26.21 seconds runtime

ainv_df

	Date ▲	Open ▲	High ▲	Low ▲	Close ▲	Volume ▲	OpenInt ▲
1	2005-02-25	7.9331	8.0665	7.8034	8.0665	632096	0
2	2005-02-28	8.0756	8.1248	7.8524	7.8615	750398	0
3	2005-03-01	7.8673	8.1101	7.8524	8.0756	977229	0
4	2005-03-02	8.0385	8.1335	8.0385	8.0625	837318	0
5	2005-03-03	8.0625	8.148	8.0287	8.0996	541465	0
6	2005-03-04	8.0707	8.148	7.9183	8.1429	1219409	0
7	2005-03-07	8.1429	8.2104	8.0756	8.1248	587457	0

↓ 3,201 rows | 26.21 seconds runtime

ale_df

	Date ▲	Open ▲	High ▲	Low ▲	Close ▲	Volume ▲	OpenInt ▲
1	2005-02-25	30.717	31.104	30.6	31.087	214817	0
2	2005-02-28	31.073	31.087	30.652	30.7	180957	0
3	2005-03-01	30.659	30.946	30.606	30.946	223738	0
4	2005-03-02	30.908	31.179	30.884	31.073	238730	0
5	2005-03-03	31.073	31.211	30.987	31.195	224381	0
6	2005-03-04	31.335	31.836	31.288	31.836	227100	0
7	2005-03-07	31.875	33.788	31.836	33.663	676255	0

↓ 3,201 rows | 26.21 seconds runtime

- **Ερώτημα 1ο:** Ποιος ήταν ο μέσος όρος για τις τιμές του ανοίγματος, κλεισίματος και όγκου συναλλαγών για κάθε ημερολογιακό μήνα για κάθε μετοχή;

Αναζητούμε μέσους όρους ανοίγματος, κλεισίματος και όγκου συναλλαγών για κάθε μήνα. Άρα στο dataframe `agn_df` ομαδοποιούμε τις τιμές μέσω της `group_by` με βάση τον μήνα του πεδίου `Date`. Χρησιμοποιούμε επίσης, την συνάρτηση `agg` για να εκτελέσουμε πολλαπλούς υπολογισμούς μέσω των `Open`, `Close` και `Volume`.

Με τον ίδιο ακριβώς τρόπο υπολογίζονται οι αντίστοιχες μέσες τιμές για τις μετοχές `ainv` και `ale`.

Cmd 5

Ποιος ήταν ο μέσος όρος για τις τιμές του ανοίγματος, κλεισίματος και όγκου συναλλαγών για κάθε ημερολογιακό μήνα για κάθε μετοχή

Cmd 6

```
1 # Μετοχή agn
2 agn_avg = agn_df.groupby(month("Date").alias("Month")).agg(avg("Open").alias("Average Open"), avg("Close").alias("Average Close"), avg("Volume").alias("Average Volume")).orderBy("Month")
3
4 print("Ποιος ήταν ο μέσος όρος για τις τιμές του ανοίγματος, κλεισίματος και όγκου συναλλαγών για κάθε ημερολογιακό μήνα για την μετοχή agn;")
5 agn_avg.show(truncate=False)
```

```
6
7 # Μετοχή ainv
8 ainv_avg = ainv_df.groupby(month("Date").alias("Month")).agg(avg("Open").alias("Average Open"), avg("Close").alias("Average Close"), avg("Volume").alias("Average Volume")).orderBy("Month")
9
10 print("Ποιος ήταν ο μέσος όρος για τις τιμές του ανοίγματος, κλεισίματος και όγκου συναλλαγών για κάθε ημερολογιακό μήνα για την μεταχολή ainv;")
11 ainv_avg.show(truncate=False)
12
13 # Μετοχή ale
14 ale_avg = ale_df.groupby(month("Date").alias("Month")).agg(avg("Open").alias("Average Open"), avg("Close").alias("Average Close"), avg("Volume").alias("Average Volume")).orderBy("Month")
15
16 print("Ποιος ήταν ο μέσος όρος για τις τιμές του ανοίγματος, κλεισίματος και όγκου συναλλαγών για κάθε ημερολογιακό μήνα για την μεταχολή ale;")
17 ale_avg.show(truncate=False)
```

Απάντηση

Ποιος ήταν ο μέσος όρος για τις τιμές του ανοίγματος, κλεισίματος και όγκου συναλλαγών για κάθε ημερολογιακό μήνα για την μεταχολή ainv;				
Month	Average Open	Average Close	Average Volume	
1	101.83225246590216	101.80168585941709	1796786.0	
2	106.105923828125	106.37493228149414	2018339.112	
3	108.97477564895362	108.91759996916119	1954400.0771929824	
4	104.96801150023047	104.85799636413802	2048475.3246268656	
5	106.58948712435635	106.51974534468218	2020206.469090909	
6	110.05557827268328	110.13075361933026	1611504.9321428572	
7	113.9869779440073	114.04540697034899	1486405.2857142857	
8	112.94372663530923	112.76380307286668	1592136.8719723183	
9	113.42711665935086	113.29473668292053	1720100.3007518798	
10	109.94360694885253	109.83696154544228	2045412.403508772	
11	105.93957639400196	105.72473086481509	2363413.675889328	
12	107.0282400236355	107.06868900088813	1565874.4606299212	

Ποιος ήταν ο μέσος όρος για τις τιμές του ανοίγματος, κλεισίματος και όγκου συναλλαγών για κάθε ημερολογιακό μήνα για την μεταχολή ainv;				
Month	Average Open	Average Close	Average Volume	
1	5.960922838741318	5.9578900446041	2664324.3029045644	
2	5.910714666391241	5.914387079148457	3277507.870689655	
3	6.044150871561285	6.036955775085248	3266939.4210526315	
4	6.208691410609146	6.210719027625981	2431237.593283582	
5	6.2512025330283425	6.252906892082908	2611925.7163636363	
6	6.227297857829503	6.216372852666037	2798179.3964285715	
7	6.111186066826622	6.118620138901931	2189656.0036630034	
8	6.283973707872279	6.294057775946224	2754273.910034602	
9	6.352289081516123	6.345983075020008	2706609.6917293235	
10	6.21246140546966	6.203042460324472	2141400.1192982458	
11	6.114790127682592	6.1033130511936	2523151.9881422925	
12	5.926174418194088	5.918955116760074	2916861.6377952755	

Ποιος ήταν ο μέσος όρος για τις τιμές του ανοίγματος, κλεισίματος και όγκου συναλλαγών για κάθε ημερολογιακό μήνα για την μεταχολή ale;				
Month	Average Open	Average Close	Average Volume	
1	37.59970537557641	37.633916973573044	223401.15352697094	
2	38.356318876661106	38.34859057130485	240881.875	
3	38.04251227127878	38.07297179238838	253724.5649122807	
4	38.59285437882836	38.6146268986944	240209.06343283583	
5	39.37950182134455	39.382316367409445	244737.14909090908	
6	39.89017846924918	39.91864985738482	257443.88214285715	
7	40.26568859924764	40.257758206936906	209357.9010989011	
8	40.10728026261379	40.0871074108929	245987.4429065744	
9	39.87034198933078	39.861116545540945	226992.70676691728	
10	40.066157906515556	40.064231524551126	235927.1754385965	
11	38.16249403369285	38.125442693356	225893.2648221344	
12	37.39472042293999	37.40590548327589	234291.1968503937	

- **Ερώτημα 2ο:** Για πόσες ημέρες ήταν η τιμή του ανοίγματος της κάθε μετοχής πάνω από 35 δολάρια;

Για το ερώτημα αυτό μας ενδιαφέρει μόνο η στήλη των ημερομηνιών και της τιμής του ανοίγματος σε κάθε dataframe. Χρησιμοποιούμε, λοιπόν, την filter στην col("Open") βάζοντας το κριτήριο του να είναι μεγαλύτερη η τιμή από 35. Για να βρούμε το πλήθος των ημερών με τιμή ανοίγματος μεγαλύτερη από 35 βάζουμε στο τέλος της εντολής και την συνάρτηση count(). Η ίδια διαδικασία ακολουθείται και για τις τρεις μετοχές, δηλαδή και στα τρία αρχικά dataframes.

Cmd 7

Για πόσες ημέρες ήταν η τιμή του ανοίγματος της κάθε μετοχής πάνω από 35 δολάρια;

Cmd 8



```
1 agn_open = agn_df.filter(col("Open") > 35.0000).count()
2 print("Για πόσες ημέρες ήταν η τιμή του ανοίγματος της μετοχής agn πάνω από 35 δολάρια;")
3 display(agn_open)
4 print()
5
6 ainv_open = ainv_df.filter(col("Open") > 35.0000).count()
7 print("Για πόσες ημέρες ήταν η τιμή του ανοίγματος της μετοχής ainv πάνω από 35 δολάρια;")
8 display(ainv_open)
9 print()
10
11 ale_open = ale_df.filter(col("Open") > 35.0000).count()
12 print("Για πόσες ημέρες ήταν η τιμή του ανοίγματος της μετοχής ale πάνω από 35 δολάρια;")
13 display(ale_open)
```

Απάντηση

Για πόσες ημέρες ήταν η τιμή του ανοίγματος της μετοχής agn πάνω από 35 δολάρια;
2071

Για πόσες ημέρες ήταν η τιμή του ανοίγματος της μετοχής ainv πάνω από 35 δολάρια;
0

Για πόσες ημέρες ήταν η τιμή του ανοίγματος της μετοχής ale πάνω από 35 δολάρια;
1667

- **Ερώτημα 3ο:** Ποιες ήταν οι μέρες με την υψηλότερη τιμή στο άνοιγμα και ποιες με την υψηλότερη τιμή στον όγκο συναλλαγών για κάθε μετοχή;

Για τον εντοπισμό της ημέρας με την υψηλότερη τιμή ανοίγματος χρησιμοποιούμε την συνάρτηση max στην στήλη "Open" και της δίνουμε την ονομασία "Maximum Open". Ενώνουμε μέσω του join την τιμή που βρήκαμε με το αρχικό dataframe, δηλαδή η τιμή αυτή τοποθετείται ως επιπλέον στήλη και αντιστοιχεί σε συγκεκριμένη ημερομηνία. Τέλος, επιλέγουμε τα πεδία "Date" και "Maximum Open", πεδία που μας δίνουν την ζητούμενη ημερομηνία και την τιμή της.

Για τον εντοπισμό της ημέρας με την υψηλότερη τιμή στον όγκο συναλλαγών ακολουθούμε μία παρόμοια διαδικασία. Στην στήλη volume του αρχικού dataframe εφαρμόζουμε την συνάρτηση max, ώστε να βρούμε την μέγιστη τιμή. Μέσω της join, η μέγιστη αυτή τιμή που την ονομάσαμε "Maximum Volume" προστίθεται στο dataframe και αντιστοιχεί στην ημερομηνία της. Έτσι, κάνοντας select τα πεδία "Date" και "Maximum Volume" (χωρίς δεκαδικά ψηφία), παίρνουμε την απάντηση.

Οι παραπάνω διαδικασίες αφορούν την εύρεση της υψηλότερης τιμής των πεδίων "Open" και "Volume" και για τις τρεις μετοχές.

Cmd 9

Ποιες ήταν οι μέρες με την υψηλότερη τιμή στο άνοιγμα και ποιες με την υψηλότερη τιμή στον όγκο συναλλαγών για κάθε μετοχή;

Cmd 10

```
1 # agn: Εντοπίζουμε την μέγιστη τιμή του Open
2 agn_max_open = (
3     agn_df
4     .agg(max("Open").alias("Maximum Open"))
5     .join(agn_df, col("Open") == col("Maximum Open"))
6     .select("Date", "Maximum Open")
7 )
8 print("Η μέρα με την υψηλότερη τιμή στο άνοιγμα για την μετοχή agn:")
9 agn_max_open.show(truncate=False)
10
11 # agn: Εντοπίζουμε την μέγιστη τιμή του Volume
12 agn_max_volume = (
13     agn_df
14     .agg(max("Volume").alias("Maximum Volume"))
15     .join(agn_df, col("Volume") == col("Maximum Volume"))
16     .select("Date", format_number("Maximum Volume", 0).alias("Maximum Volume"))
17 )
18 print("Η μέρα με την υψηλότερη τιμή στον όγκο συναλλαγών για την μετοχή agn:")
19 agn_max_volume.show(truncate=False)
20
21 # ain: Εντοπίζουμε την μέγιστη τιμή του Open
22 ainv_max_open = (
23     ainv_df
24     .agg(max("Open").alias("Maximum Open"))
25     .join(ainv_df, col("Open") == col("Maximum Open"))
26     .select("Date", "Maximum Open")
27 )
28 print("Η μέρα με την υψηλότερη τιμή στο άνοιγμα για την μετοχή ain:")
29 ainv_max_open.show(truncate=False)
30
31 # ain: Εντοπίζουμε την μέγιστη τιμή του Volume
32 ainv_max_volume = (
33     ainv_df
34     .agg(max("Volume").alias("Maximum Volume"))
35     .join(ainv_df, col("Volume") == col("Maximum Volume"))
36     .select("Date", format_number("Maximum Volume", 0).alias("Maximum Volume"))
37 )
38 print("Η μέρα με την υψηλότερη τιμή στον όγκο συναλλαγών για την μετοχή ain:")
39 ainv_max_volume.show(truncate=False)
```

```

40
41 # ale: Εντοπίζουμε την μέγιστη τιμή του Open
42 ale_max_open = (
43     ale_df
44     .agg(max("Open").alias("Maximum Open"))
45     .join(ale_df, col("Open") == col("Maximum Open"))
46     .select("Date", "Maximum Open")
47 )
48 print("Η μέρα με την υψηλότερη τιμή στο άνοιγμα για την μετοχή ale:")
49 ale_max_open.show(truncate=False)
50
51 # ale: Εντοπίζουμε την μέγιστη τιμή του Volume
52 ale_max_volume = (
53     ale_df
54     .agg(max("Volume").alias("Maximum Volume"))
55     .join(ale_df, col("Volume") == col("Maximum Volume"))
56     .select("Date", format_number("Maximum Volume", 0).alias("Maximum Volume"))
57 )
58 print("Η μέρα με την υψηλότερη τιμή στον όγκο συναλλαγών για την μετοχή ale:")
59 ale_max_volume.show(truncate=False)

```

Απάντηση

Η μέρα με την υψηλότερη τιμή στο άνοιγμα για την μετοχή agn:

Date	Maximum Open
2015-07-30	334.08

Η μέρα με την υψηλότερη τιμή στον όγκο συναλλαγών για την μετοχή agn:

Date	Maximum Volume
2016-04-05	36,807,460

Η μέρα με την υψηλότερη τιμή στο άνοιγμα για την μετοχή ainv:

Date	Maximum Open
2007-02-20	11.474

Η μέρα με την υψηλότερη τιμή στον όγκο συναλλαγών για την μετοχή ainv:

Date	Maximum Volume
2014-02-28	57,522,364

Η μέρα με την υψηλότερη τιμή στο άνοιγμα για την μετοχή ale:

Date	Maximum Open
2017-11-01	80.0

Η μέρα με την υψηλότερη τιμή στον όγκο συναλλαγών για την μετοχή ale:

Date	Maximum Volume
2014-02-27	2,118,295

- **Ερώτημα 4ο:** Ποιες ήταν οι χρονιές που κάθε μετοχή σημείωσε την υψηλότερη τιμή στο άνοιγμα και ποιες και τη χαμηλότερη τιμή στο κλείσιμο;

Αρχικά, προσθέτουμε μία επιπρόσθετη στήλη στο dataframe, η οποία ονομάζεται year και δείχνει το έτος κάθε μέτρησης. Μέσω της max("Open") βρίσκουμε την μέγιστη τιμή του ανοίγματος της μετοχής. Εδώ το agg χρησιμοποιείται για να γίνουν ταυτόχρονα το max και το alias. Χρησιμοποιώντας την join, η "Maximum Open" συνδέεται με τα αρχικά δεδομένα της μετοχής για την συγκεκριμένη ημερομηνία. Αφού κάνουμε select μόνο το πεδίο year, παίρνουμε ως απάντηση την χρονιά την οποία καταγράφηκε η μέγιστη τιμή ανοίγματος.

Αντίστοιχα για την τιμή του close χρησιμοποιούμε την min("Close"), ώστε να βρούμε την μικρότερη τιμή κλεισίματος. Εδώ το agg χρησιμοποιείται για να γίνουν ταυτόχρονα το max και το alias. Μέσω της join το Minimum Open που εντοπίσαμε συνδέεται με τα αρχικά δεδομένα, ενώ με την select("Year") παίρνουμε ως απάντηση μόνο την επιθυμητή χρονιά.

Οι παραπάνω διαδικασίες είναι ακριβώς ίδιες και για τις τρεις μετοχές.

Ποιες ήταν οι χρονιές που κάθε μετοχή σημείωσε την υψηλότερη τιμή στο άνοιγμα και ποιες και τη χαμηλότερη τιμή στο κλείσιμο;

```
1 # agn: Εντοπίζουμε την χρονιά της μέγιστης τιμής του Open
2 agn_df = agn_df.withColumn("Year", year("Date"))
3
4 agn_max_open_year = (
5     agn_df
6     .agg(max("Open").alias("Maximum Open"))
7     .join(agn_df, (col("Open") == col("Maximum Open")))
8     .select("Year")
9 )
10 max_open_year_agn = agn_max_open_year.select("Year").first()[0]
11 print(f"Η χρονιά με την υψηλότερη τιμή στο άνοιγμα για την μετοχή agn: {max_open_year_agn}\n")
12
13 # agn: Εντοπίζουμε την χρονιά της ελάχιστης τιμής του Close
14 agn_min_close_year = (
15     agn_df
16     .agg(min("Close").alias("Minimum Close"))
17     .join(agn_df, (col("Close") == col("Minimum Close")))
18     .select("Year")
19 )
20 min_close_year_agn = agn_min_close_year.select("Year").first()[0]
21 print(f"Η χρονιά με την χαμηλότερη τιμή στο άνοιγμα για την μετοχή agn: {min_close_year_agn}\n")
```

```

23 # ainv: Εντοπίζουμε την χρονιά της μέγιστης τιμής του Open
24 ainv_df = ainv_df.withColumn("Year", year("Date"))
25
26 ainv_max_open_year = (
27     ainv_df
28     .agg(max("Open").alias("Maximum Open"))
29     .join(ainv_df, (col("Open") == col("Maximum Open")))
30     .select("Year")
31 )
32 max_open_year_ainv = ainv_max_open_year.select("Year").first()[0]
33 print(f"Η χρονιά με την υψηλότερη τιμή στο άνοιγμα για την μετοχή ainv: {max_open_year_ainv}\n")
34
35 # ainv: Εντοπίζουμε την χρονιά της ελάχιστης τιμής του Close
36 ainv_min_close_year = (
37     ainv_df
38     .agg(min("Close").alias("Minimum Close"))
39     .join(ainv_df, (col("Close") == col("Minimum Close")))
40     .select("Year")
41 )
42 min_close_year_ainv = ainv_min_close_year.select("Year").first()[0]
43 print(f"Η χρονιά με την χαμηλότερη τιμή στο άνοιγμα για την μετοχή ainv: {min_close_year_ainv}\n")
44
45 # ale: Εντοπίζουμε την χρονιά της μέγιστης τιμής του Open
46 ale_df = ale_df.withColumn("Year", year("Date"))
47
48 ale_max_open_year = (
49     ale_df
50     .agg(max("Open").alias("Maximum Open"))
51     .join(ale_df, (col("Open") == col("Maximum Open")))
52     .select("Year")
53 )
54 max_open_year_ale = ale_max_open_year.select("Year").first()[0]
55 print(f"Η χρονιά με την υψηλότερη τιμή στο άνοιγμα για την μετοχή ale: {max_open_year_ale}\n")
56
57 # ale: Εντοπίζουμε την χρονιά της ελάχιστης τιμής του Close
58 ale_min_close_year = (
59     ale_df
60     .agg(min("Close").alias("Minimum Close"))
61     .join(ale_df, (col("Close") == col("Minimum Close")))
62     .select("Year")
63 )
64 min_close_year_ale = ale_min_close_year.select("Year").first()[0]
65 print(f"Η χρονιά με την χαμηλότερη τιμή στο άνοιγμα για την μετοχή ale: {min_close_year_ale}\n")

```

Απάντηση

Η χρονιά με την υψηλότερη τιμή στο άνοιγμα για την μετοχή agn: 2015

Η χρονιά με την χαμηλότερη τιμή στο άνοιγμα για την μετοχή agn: 2008

Η χρονιά με την υψηλότερη τιμή στο άνοιγμα για την μετοχή a1nv: 2007

Η χρονιά με την χαμηλότερη τιμή στο άνοιγμα για την μετοχή a1nv: 2009

Η χρονιά με την υψηλότερη τιμή στο άνοιγμα για την μετοχή ale: 2017

Η χρονιά με την χαμηλότερη τιμή στο άνοιγμα για την μετοχή ale: 2009

Θέμα 3

Στο θέμα αυτό καλούμαστε να επεξεργαστούμε δεδομένα από ένα excel αρχείο, το οποίο αφορά το πλήθος τουριστών που διανυκτέρευσαν σε 36 ευρωπαϊκές χώρες κάθε χρονιά από το 2006 έως το 2015.

Αρχικά, εισάγουμε τις απαραίτητες βιβλιοθήκες. Όπως και στα προηγούμενα εισάγεται το module **functions** και ένα σεν από τις συναρτήσεις που περιέχει: **col, avg, lit, concat_ws, udf, array, when, regexp_replace, expr, year, min**. Οι συναρτήσεις αυτές χρησιμοποιούνται αργότερα για τους υπολογισμούς που απαιτούν τα ερωτήματα. Επίσης, εισάγεται το module **pyspark.sql.types** με τις κλάσεις **StructType, StructField, StringType, DoubleType, FloatType** ώστε να διαχειριστούμε τα στοιχεία που περιέχει το excel file.

Επίσης, εισάγεται η κλάση Window που βοηθά στην διαχείριση παραθύρων σε ένα dataframe. Στην ουσία βοηθά στον ορισμό υποσυνόλων εντός του dataframe βάσει στηλών και ταξινόμησης.

Cmd 1

Απαραίτητες Βιβλιοθήκες

Cmd 2

```
1 from pyspark.sql import functions as F
2 from pyspark.sql.window import Window
3 from pyspark.sql.functions import col, avg, lit, concat_ws, udf, array, when, regexp_replace,
  expr, year, min
4 from pyspark.sql.types import StructType, StructField, StringType, DoubleType, FloatType
```

Για να διαβάσουμε και να διαχειριστούμε το αρχείο **tour_occ_ninat.xlsx** χρειάστηκε αρχικά να εγκαταστήσουμε στο databricks την βιβλιοθήκη **spark-excel_2.12-0.13.5**. Αυτή περιέχει τις κατάλληλες συναρτήσεις, ώστε να διαβάσουμε το αρχείο και να δημιουργήσουμε το dataframe **eurostat_df**.

Ορίζουμε, το path του αρχείου `tour_occ_ninat.xlsx`, αφού το φορτώσαμε στο `databricks`. Για το διάβασμά του χρησιμοποιούμε την βιβλιοθήκη `com.crealytics.spark.excel`. Καθορίζουμε τη μορφή του αρχείου με την `format` και δηλώνουμε με `header true` πως τα στοιχεία της πρώτης γραμμής θα μπουν ως `labels` στο `dataframe`. Επειδή τα στοιχεία με τους πληθυσμούς που διανυκτέρευσαν στις ευρωπαϊκές χώρες δεν βρίσκονται στις πρώτες σειρές του αρχείου χρησιμοποιούμε το `option("dataAddress", "...")`. Στο δεύτερο πεδίο επιλέξαμε το `sheet` που μας αφορά και το `range` των κελιών που μας αφορούν.

```
1 # Path για το file tour_occ_ninat.xlsx
2 eurostat_path = "dbfs:/FileStore/tables/tour_occ_ninat.xlsx"
3
4 # Διάβασμα του αρχείου Eurostat με τη βιβλιοθήκη com.crealytics.spark.excel
5 eurostat_df = (
6     spark.read.format("com.crealytics.spark.excel")
7     .option("header", "true")
8     .option("inferSchema", "false")
9     .option("dataAddress", "'Sheet1'!A9:K45")
10    .load(eurostat_path)
11 )
12
13 display(eurostat_df)
```

eurostat_df

	GEO/TIME	2006	2007	2008	2009	2010	2011	2012	2013	2014	2015
1	Belgium	16,039,090	16,271,311	16,360,702	15,451,017	16,169,676	16,723,867	16,432,646	16,511,721	17,068,872	18,852,087
2	Bulgaria	11,944,694	12,006,786	11,791,454	9,460,922	10,547,112	12,426,723	13,451,440	14,370,426	14,077,798	13,352,281
3	Czech Republic	20,090,348	20,610,186	19,987,022	17,746,893	18,365,947	19,424,839	21,793,985	22,144,896	22,110,112	23,286,515
4	Denmark	9,453,026	9,327,579	8,918,197	8,299,403	8,981,992	9,491,137	9,608,124	9,914,273	10,608,119	11,171,416
5	Germany (until 1990 former territory of the FRG)	52,947,373	54,485,379	56,239,679	54,096,574	59,658,760	63,081,467	68,161,503	71,191,942	74,805,253	78,827,773

Στο `eurostat_df` οι τιμές εισέρχονται ως `strings`. Για τον σωστό υπολογισμό στα ερωτήματα που ακολουθούν μετατρέπουμε τα στοιχεία από `String` τύπο σε `Float`. Επίσης, αντικαθιστούμε τις `null` τιμές με μηδενικά.

```
1 # Μετατροπή των στηλών με τους αριθμούς από string σε float
2 float_clm = ["2006", "2007", "2008", "2009", "2010", "2011", "2012", "2013", "2014", "2015"]
3 for col_name in float_clm:
4     eurostat_df = eurostat_df.withColumn(col_name, regexp_replace(col(col_name), ",", "").cast("float"))
5
6 #eurostat_df.dtypes
7
8 # Αντικατάσταση των null τιμών με 0
9 eurostat_df_new = eurostat_df.fillna(0)
10 display(eurostat_df_new)
```

- **Ερώτημα 1ο: Ποιος ήταν ο μέσος όρος διανυκτερεύσεων για κάθε χώρα για το χρονικό διάστημα 2007-2014;**

Αρχικά, επιλέγουμε μόνο τις στήλες από το 2007 μέχρι το 2014, καθώς είναι οι χρονιές που μας αφορούν. Για τον μέσο όρο των διανυκτερεύσεων προκύπτει μία νέα στήλη με όνομα `Average_Overnights_2007-2014`. Οι τιμές υπολογίζονται αθροίζοντας το πλήθος κάθε έτους και διαιρώντας με το μήκος του `selected columns` που είναι 8. Για την απάντηση επιλέγουμε τα πεδία "GEO/TIME" και "Average_Overnights_2007-2014" για να πάρουμε το όνομα κάθε χώρας και τον αντίστοιχο μέσο όρο.

Cmd 7

Ποιος ήταν ο μέσος όρος διανυκτερεύσεων για κάθε χώρα για το χρονικό διάστημα 2007-2014;

Cmd 8

```
1 # Επιλέγουμε τις στήλες από το 2007 μέχρι το 2014
2 selected_columns = eurostat_df_new.columns[2:10]
3
4 # Υπολογισμός Μέσου Όρου ανά γραμμή
5 eurostat_df_1 = eurostat_df_new.withColumn(
6     "Average_Overnights_2007-2014",
7     sum(eurostat_df_new[col] for col in selected_columns) / len(selected_columns),
8 ).select("GEO/TIME", "Average_Overnights_2007-2014")
9
10 print("Ποιος ήταν ο μέσος όρος διανυκτερεύσεων για κάθε χώρα για το χρονικό διάστημα
11       2007-2014;")
12 display(eurostat_df_1)
```

Απάντηση

	GEO/TIME ▲	Average_Overnights_2007-2014 ▲
1	Belgium	16373726
2	Bulgaria	12266582
3	Czech Republic	20272984
4	Denmark	9393603
5	Germany (until 1990 former territory of the FRG)	62715068
6	Estonia	3399133
7	Ireland	4248434.5
8	Greece	60361192
9	Spain	232243712
10	France	118260272
11	Croatia	43389740
12	Italy	172326816
13	Cyprus	12894811
14	Latvia	2233122.75
15	Lithuania	2088391.125
16	Luxembourg	2194231.75
17	Hungary	10550699
18	Malta	7568818.5

19	Netherlands	28351538
20	Austria	74799824
21	Poland	11090723
22	Portugal	28976308
23	Romania	3246491.5
24	Slovenia	5228881.5
25	Slovakia	4234543.5
26	Finland	5450928
27	Sweden	11371281
28	United Kingdom	65383004
29	Iceland	2609967
30	Liechtenstein	149480.875
31	Norway	7966946
32	Switzerland	0
33	Montenegro	1360802.625
34	Former Yugoslav Republic of Macedonia, the	625334.375
35	Serbia	715064.375
36	Turkey	23396916

- **Ερώτημα 2ο: Για πόσες και ποιες χρονιές ήταν ο αριθμός διανυκτερεύσεων της χώρας Ελλάδα υψηλότερος από 5 άλλες ευρωπαϊκές χώρες (Βέλγιο, Βουλγαρία, Τσεχία, Δανία, Γερμανία);**

Στο ερώτημα αυτό, οι χώρες τις οποίες επιλέξαμε είναι οι εξής: **Βέλγιο, Βουλγαρία, Τσεχία, Δανία, Γερμανία**. Αρχικά, δημιουργούμε μία νέα στήλη την 'row number' που περιέχει τον αύξοντα αριθμό κάθε σειράς.

- Η λίστα 'grammes' περιέχει τα νούμερα των χωρών που μας αφορούν.
- Η 'selected_columns' είναι μία λίστα που περιέχει μόνο τις στήλες των ετών.
- Το dataframe 'greeced_df' περιέχει μόνο τις διανυκτερεύσεις για την Ελλάδα.
- Το 'eurostat_df_2' περιέχει τις διανυκτερεύσεις για όσες χώρες έχουν αριθμό που ανήκει στην λίστα 'grammes'.

Στην συνέχεια, για να γίνει η σύγκριση μεταξύ των διανυκτερεύσεων της Ελλάδας και των άλλων χωρών χρησιμοποιούμε μία for και έναν counter που δείχνει το πλήθος των ετών που οι διανυκτερεύσεις της Ελλάδας ήταν περισσότερες από τις υπόλοιπες πέντε χώρες.

Σε κάθε επανάληψη παίρνουμε το πλήθος των διανυκτερεύσεων της Ελλάδας και θέτουμε το flag που λέει ότι αυτή η τιμή είναι υψηλότερη από όλες ίση με true. Έπειτα αυτή η τιμή συγκρίνεται με την αντίστοιχη τιμή κάθε χώρας από τις 5. Αν σε όλες τις συγκρίσεις είναι μεγαλύτερη η τιμή της Ελλάδας τότε ο counter αυξάνεται κατά 1, αλλιώς γίνεται break και ξεκινάει η επανάληψη για το επόμενο έτος, παίρνοντας την τιμή της Ελλάδας για το επόμενο έτος.

Για πόσες και ποιες χρονιές ήταν ο αριθμός διανυκτερεύσεων της χώρας Ελλάδα υψηλότερος από 5 άλλες ευρωπαϊκές χώρες (Βέλγιο, Βουλγαρία, Τσεχία, Δανία, Γερμανία)

```
1 # Κρατάμε στο dataframe μόνο τις χώρες που μας αφορούν
2 eurostat_df_2 = eurostat_df_new.withColumn("row_number", F.row_number().over(Window.orderBy(F.
monotonically_increasing_id()))
3 #display(eurostat_df_2)
4
5 grammes = [1, 2, 3, 4, 5, 8]
6 selected_columns = eurostat_df_new.columns[1:]
7
8 # Διανυκτερεύσεις στην Ελλάδα (2006-2015)
9 greece_df = eurostat_df_2.filter(col("GEO/TIME") == "Greece").select(selected_columns)
10
11 # Διανυκτερεύσεις σε Βέλγιο, Βουλγαρία, Τσεχία, Δανία, Γερμανία (2006-2015)
12 eurostat_df_2 = eurostat_df_2.filter(col("row_number").isin(grammes)).drop("row_number")
13
14 counter = 0
15
```

```

16 for column in selected_columns:
17     greece_value = eurostat_df_2.filter(col("GEO/TIME") == "Greece").select(column).first()[column]
18     greece_largest = True
19
20     for row_number in range(1, eurostat_df_2.count()):
21         current_value = eurostat_df_2.filter(col("row_number") == row_number).select(column).first()[column]
22         if greece_value <= current_value:
23             greece_largest = False
24             break
25
26     if greece_largest:
27         counter = counter + 1
28         print("Περισσότερες διανυκτερεύσεις τουριστών στην Ελλάδα την χρονιά: ", column)
29
30 print("Ο συνολικός αριθμός ετών που ο αριθμός διανυκτερεύσεων Ελλάδας ήταν υψηλότερος από αυτόν του Βελγίου, της Βουλγαρίας, της Τσεχίας, της Δανίας και της Γερμανίας είναι: ", counter)

```

Απάντηση:

Περισσότερες διανυκτερεύσεις τουριστών στην Ελλάδα την χρονιά: 2009

Περισσότερες διανυκτερεύσεις τουριστών στην Ελλάδα την χρονιά: 2011

Ο συνολικός αριθμός ετών που ο αριθμός διανυκτερεύσεων Ελλάδας ήταν υψηλότερος από αυτόν του Βελγίου, της Βουλγαρίας, της Τσεχίας, της Δανίας και της Γερμανίας είναι: 2

- Ερώτημα 3ο: Ποιες ήταν οι χώρες με το μεγαλύτερο αριθμό διανυκτερεύσεων ανά έτος;

Για την εύρεση της χώρας με το μεγαλύτερο αριθμό διανυκτερεύσεων κάθε χρόνο από το 2006 μέχρι το 2015 δημιουργούμε πρώτα ένα κενό παράθυρο και μία λίστα με τις στήλες που θα μας απασχολήσουν.

Πιο αναλυτικά, επιλέγεται η πρώτη τιμή από το πεδίο 'GEO/TIME' μέσω της first, την ώρα που οι χώρες έχουν ταξινομηθεί (orderBy) κατά φθίνουσα σειρά (desc) εντός του παραθύρου με βάσει τον αριθμό των διανυκτερεύσεων τους (col(year)) για το συγκεκριμένο έτος. Αυτό μέσω της for year in selected_columns γίνεται για όλα τα έτη.

Προκύπτει ένα dataframe που περιέχει σε κάθε γραμμή την χώρα με το μεγαλύτερο πλήθος διανυκτερεύσεων και γι' αυτό χρησιμοποιούμε το limit(1) και έτσι έχουμε μόνο μία γραμμή με την κατάλληλη χώρα.

```

1  selected_columns = [str(year) for year in range(2006, 2016)]
2
3  eurostat_df_max = eurostat_df_new.select(
4      [
5          first("GEO/TIME").over(
6              Window.orderBy(col(year).desc())
7          ).alias(f"Max_overnight_{year}")
8          for year in selected_columns
9      ]
10 )
11 eurostat_df_max = eurostat_df_max.limit(1)
12
13 print("Ποιες ήταν οι χώρες με το μεγαλύτερο αριθμό διανυκτερεύσεων ανά έτος;")
14 eurostat_df_max.show(truncate=False)

```

Απάντηση:

Ποιες ήταν οι χώρες με το μεγαλύτερο αριθμό διανυκτερεύσεων ανά έτος;

Max_overnight_2006	Max_overnight_2007	Max_overnight_2008	Max_overnight_2009	Max_overnight_2010
Spain	Spain	Spain	Spain	Spain

Max_overnight_2011	Max_overnight_2012	Max_overnight_2013	Max_overnight_2014	Max_overnight_2015
Spain	Spain	Spain	Spain	Spain

- **Ερώτημα 4ο: Ποια ήταν η χρονιά που η κάθε χώρα είχε το μικρότερο αριθμό διανυκτερεύσεων σε σχέση με όλες τις υπόλοιπες ευρωπαϊκές χώρες;**

Για το τελευταίο ερώτημα θεωρήθηκε πως το ζητούμενο είναι το έτος στο οποίο κάθε χώρα κατέγραψε τον ελάχιστο αριθμό τουριστών. Έτσι, λοιπόν προσθέτουμε στο αρχικό dataframe μία νέα στήλη με όνομα 'Min_Overnights' που περιέχει το ελάχιστο πλήθος για κάθε χώρα. Αυτό το πλήθος εξάγεται από την `least(*selected_columns)` που επιλέγει το ελάχιστο κάθε γραμμής για τις επιλεγμένες στήλες.

Στην συνέχεια καλούμαστε να βρούμε την χρονιά που αντιστοιχεί σε αυτό το ελάχιστο πλήθος για κάθε χώρα. Δημιουργούμε μία νέα στήλη 'year'. Για κάθε γραμμή ελέγχουμε τις τιμές των επιλεγμένων στηλών και εντοπίζουμε σε ποια χρονιά ισχύει η ισότητα `F.col(col) == F.col("Min_Overnights")`, επιστρέφοντας την χρονιά με την μορφή ενός ακέραιου αριθμού (`F.lit(int(col))`).

Ποια ήταν η χρονιά που η κάθε χώρα είχε το μικρότερο αριθμό διανυκτερεύσεων;

```

1  # Στήλες που μας ενδιαφέρουν (2006-2015) --> selected_columns
2
3  # Ελάχιστος αριθμός διανυκτερεύσεων για κάθε χώρα
4  eurostat_df_min = eurostat_df_new.withColumn("Min_Overnights", least(*selected_columns))
5
6  # Βρίσκουμε τη χρονιά με τις λιγότερες διανυκτερεύσεις για κάθε χώρα
7  eurostat_df_min = eurostat_df_min.withColumn(
8      "Year",
9      F.coalesce(*[F.when(F.col(col) == F.col("Min_Overnights"), F.lit(int(col))).otherwise
10         (None) for col in selected_columns])
11 )
12
13 # Αποτέλεσμα με τις στήλες των Χωρών, του αριθμού ελάχιστων διανυκτερεύσεων και της
14   αντίστοιχης χρονιάς
15 result_df = eurostat_df_min.select("GEO/TIME", "Min_Overnights", "Year")
16 result_df.show(result_df.count(), truncate=False)

```

Απάντηση:

+-----+-----+					
GEO/TIME	Min_Overnights	Year	Malta	6550794.0	2009
+-----+-----+			Netherlands	2.50137E7	2009
+-----+-----+			Austria	7.0017088E7	2006
Belgium	1.5451017E7	2009	Poland	9609447.0	2009
Bulgaria	9460922.0	2009	Portugal	2.5024678E7	2009
Czech Republic	1.7746892E7	2009	Romania	2667666.0	2009
Denmark	8299403.0	2009	Slovenia	4332049.0	2006
Germany (until 1990 former territory of the FRG)	5.2947372E7	2006	Slovakia	3707327.0	2009
Estonia	2740696.0	2009	Finland	4890006.0	2009
Ireland	0.0	2007	Sweden	1.0920106E7	2008
Greece	4.305538E7	2006	United Kingdom	0.0	2013
Spain	2.00551728E8	2009	Iceland	1687744.0	2006
France	9.8705216E7	2009	Liechtenstein	110465.0	2015
Croatia	3.2858014E7	2006	Norway	7500431.0	2009
Italy	1.56861344E8	2006	Switzerland	0.0	2006
Cyprus	1.1666663E7	2009	Montenegro	0.0	2006
Latvia	1699562.0	2009	Former Yugoslav Republic of Macedonia, the	0.0	2006
Lithuania	1395899.0	2009	Serbia	0.0	2006
Luxembourg	1717130.0	2010	Turkey	0.0	2006
Hungary	9220148.0	2009			

Τα παραπάνω τμήματα κώδικα βρίσκονται εντός των τριών txt αρχείων. Το αρχείο Query1.txt περιέχει τον κώδικα του θέματος 1, το αρχείο Query2.txt περιέχει τον κώδικα του θέματος 2 και το αρχείο Query3.txt περιέχει τον κώδικα του θέματος 3.